The first three lines are the threads doing the updating, and the last two are the threads doing just reading. Each triplet is a set of the variables `x`, `y` and `z` in that order from one pass through the loop. There are a few things to notice from this output:

- The first set of values shows `x` increasing by one with each triplet, the second set has `y` increasing by one, and the third has `z` increasing by one.
- The `x` elements of each triplet only increase within a given set, as do the `y` and `z` elements, but the increments are uneven, and the relative orderings vary between all threads.
- Thread 3 doesn't see any of the updates to `x` or `y`; it sees only the updates it makes to `z`. This doesn't stop the other threads from seeing the updates to `z` mixed in with the updates to `x` and `y` though.

This is a valid outcome for relaxed operations, but it's not the only valid outcome. Any set of values that's consistent with the three variables each holding the values 0 to 10 in turn and that has the thread incrementing a given variable printing the values 0 to 9 for that variable is valid.

Understanding Relaxed Ordering

To understand how this works, imagine that each variable is a man in a cubicle with a notepad. On his notepad is a list of values. You can phone him and ask him to give you a value, or you can tell him to write down a new value. If you tell him to write down a new value, he writes it at the bottom of the list. If you ask him for a value, he reads you a number from the list.

The first time you talk to this man, if you ask him for a value, he may give you *any* value from the list he has on his pad at the time. If you then ask him for another value, he may give you the same one again or a value from farther down the list. He'll never give you a value from farther up the list. If you tell him to write down a number and then subsequently ask him for a value, he'll give you either the number you told him to write down or a number below that on the list.

Imagine for a moment that his list starts with the values 5, 10, 23, 3, 1, 2. If you ask for a value, you could get any of those. If he gives you 10, then the next time you ask he could give you 10 again, or any of the later ones, but not 5. If you call him five times, he could say "10, 10, 1, 2, 2," for example. If you tell him to write down 42, he'll add it to the end of the list. If you ask him for a number again, he'll keep telling you "42" until he has another number on his list and he feels like telling it to you.

Now, imagine your friend Carl also has this man's number. Carl can also phone him and either ask him to write down a number or ask for one, and he applies the same rules to Carl as he does to you. He has only one phone, so he can only deal with one of you at a time, so the list on his pad is a nice straightforward list. However, just because you got him to write down a new number doesn't mean he has to tell it to Carl, and vice versa. If Carl asked him for a number and was told "23," then just because you asked the man to write down 42 doesn't mean he'll tell that to Carl next time. He may tell Carl any of the numbers 23, 3, 1, 2, 42, or even the 67 that Fred told him to write down after you called. He could very well tell Carl "23, 3, 3, 1, 67" without being inconsistent with what he told you. It's like he keeps track of which number he told to whom with a little movable sticky note for each person, like in figure 5.5.